

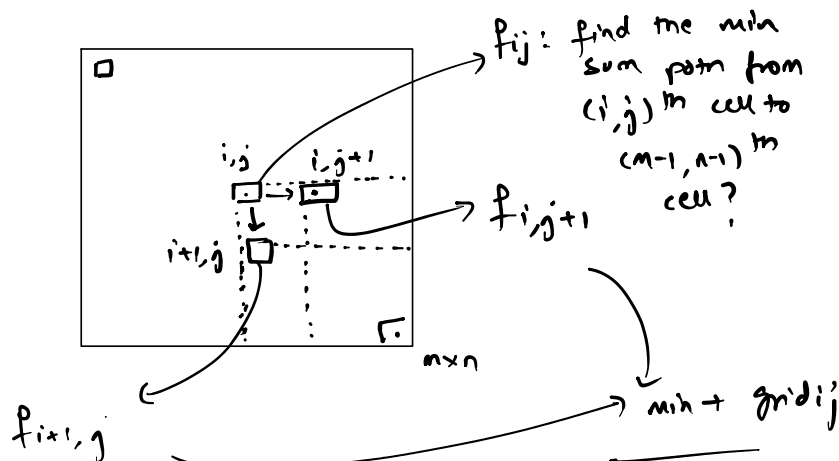
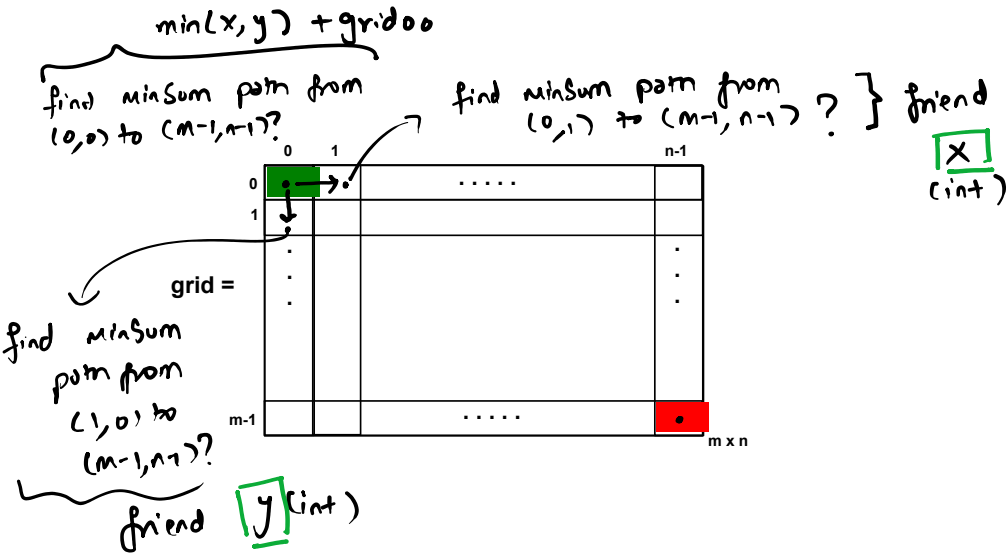
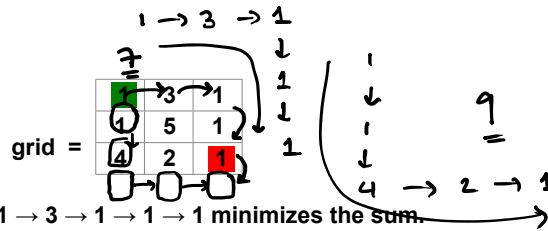
Minimum Sum Path

≥ 0

Given a $(m \times n)$ grid containing non-negative integers, design a recursive algorithm to compute the sum of numbers along the minimum sum path to reach the $(m-1, n-1)^{th}$ cell from the $(0,0)^{th}$ cell such that at each step you can either move in down or right direction.

Example :

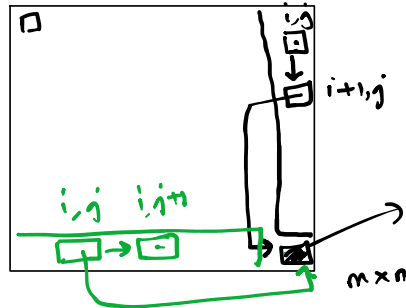
Input :



init $i=0$
 $j=0$

f_{00}

$$f_{i,j+1} + gnd_{ij}$$



$$f_{i+1,j} + gnd_{ij}$$

$$f_{m-1,n-1} = gnd_{m-1,n-1}$$

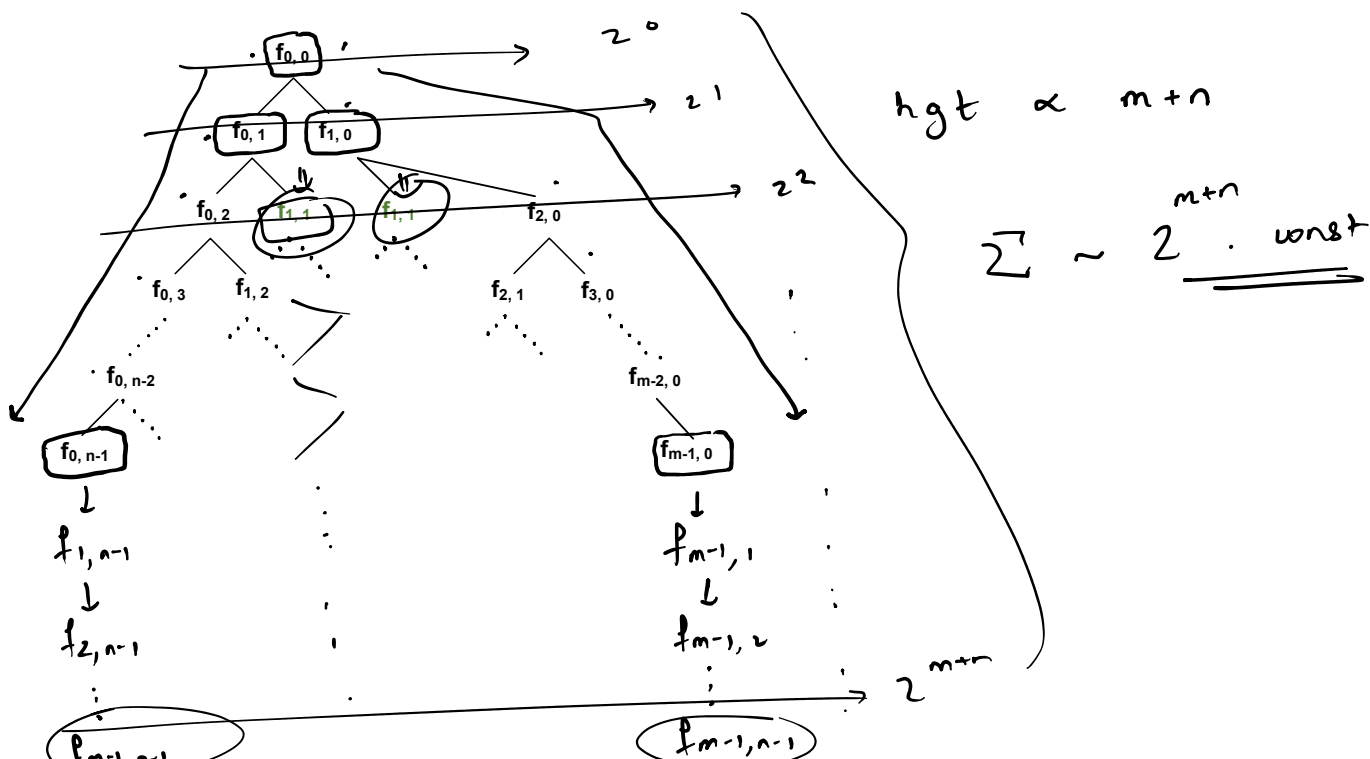
Recursive Case

Let $f(i, j)$ be a function which computes the **sum of numbers** along the **minimum** sum path to go from the $(i, j)^{th}$ cell to the $(m-1, n-1)^{th}$ cell then,

$$f(i, j) = \text{grid}[i][j] + \min [f(i, j+1), f(i+1, j)]$$

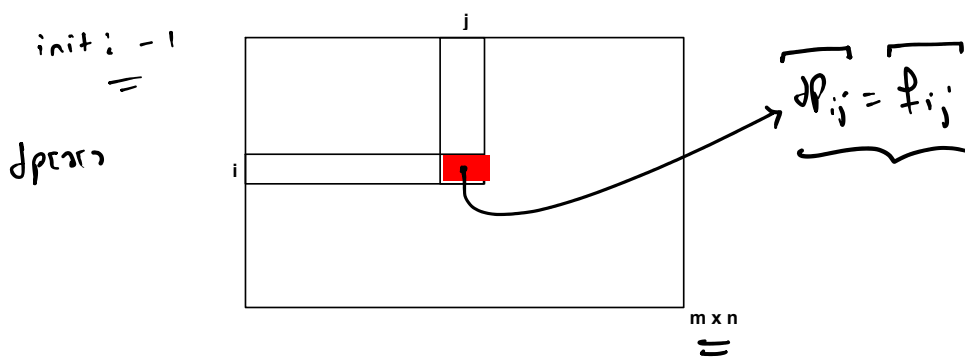
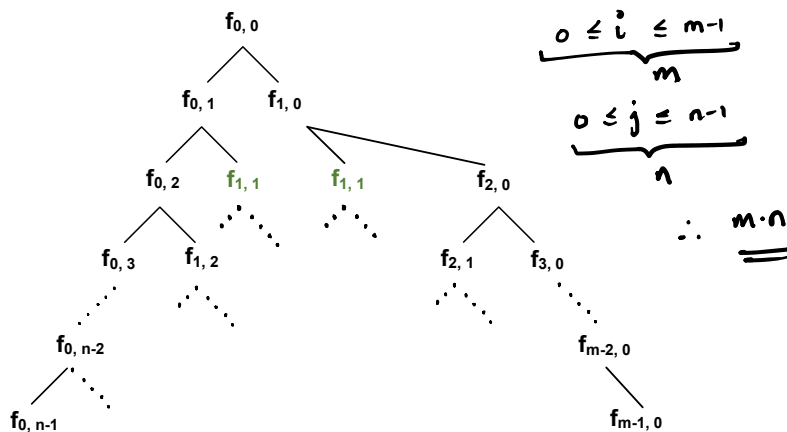
Base Case

What is the reason behind **exponential** time complexity?





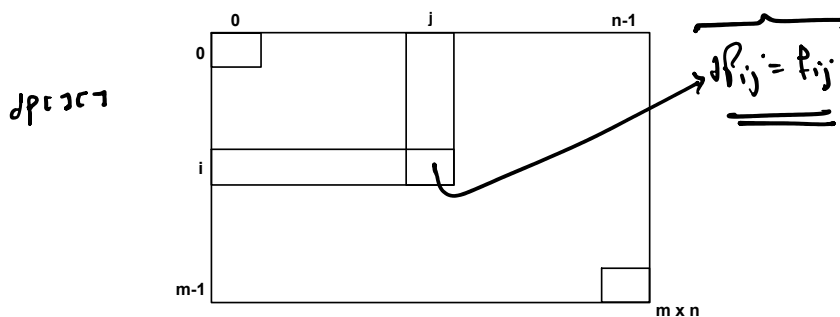
Top-Down Dynamic Programming



where,

$f(i, j)$ is a function which computes the **sum of numbers** along the **minimum sum path** to go from the $(i, j)^{\text{th}}$ to the $(m-1, n-1)^{\text{th}}$ cell.

Bottom-Up Dynamic Programming



where,

$f(i, j)$ is a function which computes the **sum of numbers** along the **minimum sum path** to go from the $(i, j)^{th}$ to the $(m-1, n-1)^{th}$ cell.

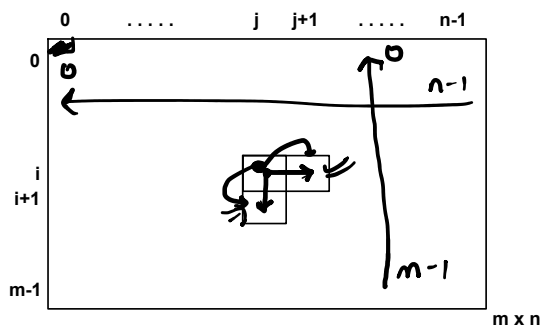
From recurrence,

$$f(i, j) = \begin{cases} \text{grid}[i][j] + f(i, j+1) & ; i = m-1 \text{ and } j \neq n-1 \\ \text{grid}[i][j] + f(i+1, j) & ; i \neq m-1 \text{ and } j = n-1 \\ \text{grid}[i][j] + \min [f(i, j+1), f(i+1, j)] & ; i \neq m-1 \text{ and } j \neq n-1 \\ \text{grid}[i][j] & ; i = m-1 \text{ and } j = n-1 \end{cases}$$

Therefore,

$$dp[i][j] = \begin{cases} \text{grid}[i][j] + dp[i][j+1] & ; i = m-1 \text{ and } j \neq n-1 \\ \text{grid}[i][j] + dp[i+1][j] & ; i \neq m-1 \text{ and } j = n-1 \\ \text{grid}[i][j] + \min (dp[i][j+1], dp[i+1][j]) & ; i \neq m-1 \text{ and } j \neq n-1 \\ \text{grid}[i][j] & ; i = m-1 \text{ and } j = n-1 \end{cases}$$

$$f_{00} = dp_{00}$$



[R][HW] Building the Min-Sum Path from $dp[][]$

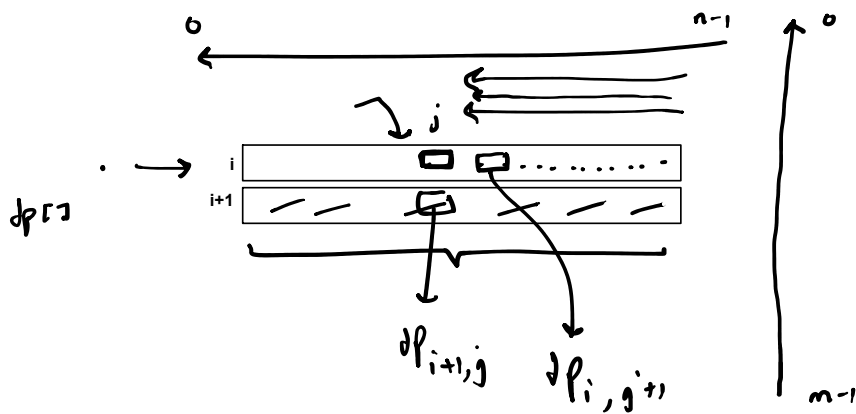
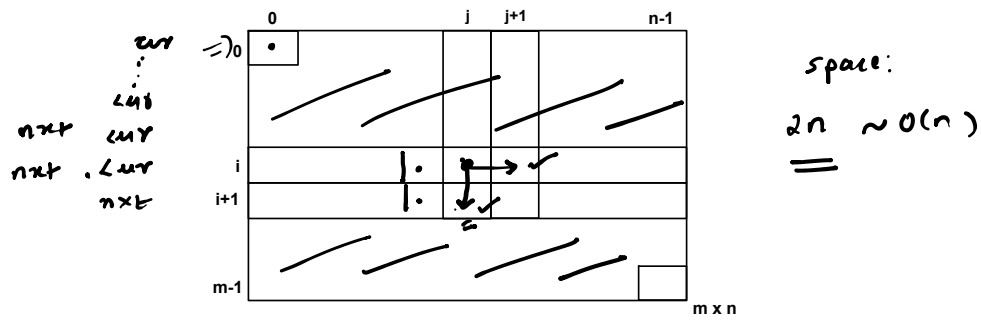
	0	1	2
0	1	3	1
1	1	5	1
2	4	2	1

grid[][]

	0	1	2
0	7	6	3
1	8	7	2
2	7	3	1

dp[][]

Space Optimized Bottom-Up Dynamic Programming



Dungeon Game

The demons had captured the princess and imprisoned her in **the bottom-right corner** of a `dungeon`. The `dungeon` consists of $m \times n$ rooms laid out in a 2D grid. Our valiant knight was initially positioned in **the top-left room** and must fight his way through `dungeon` to rescue the princess.

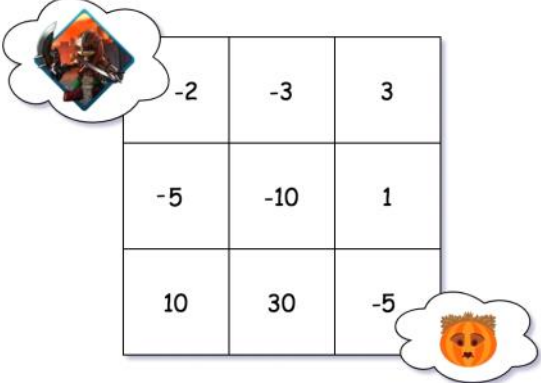
The knight has an initial health point represented by a positive integer. If at any point his health point drops to 0 or below, he dies immediately.

Some of the rooms are guarded by demons (represented by negative integers), so the knight loses health upon entering these rooms; other rooms are either empty (represented as 0) or contain magic orbs that increase the knight's health (represented by positive integers).

To reach the princess as quickly as possible, the knight decides to move only **rightward** or **downward** in each step.

Return the knight's minimum initial health so that he can rescue the princess.

Note that any room can contain threats or power-ups, even the first room the knight enters and the bottom-right room where the princess is imprisoned.



-2	-3	3
-5	-10	1
10	30	-5

dungeon =

	0	1		n-1
0				
1				
	.			.
	.			.
	.			.
m-1				

m x n

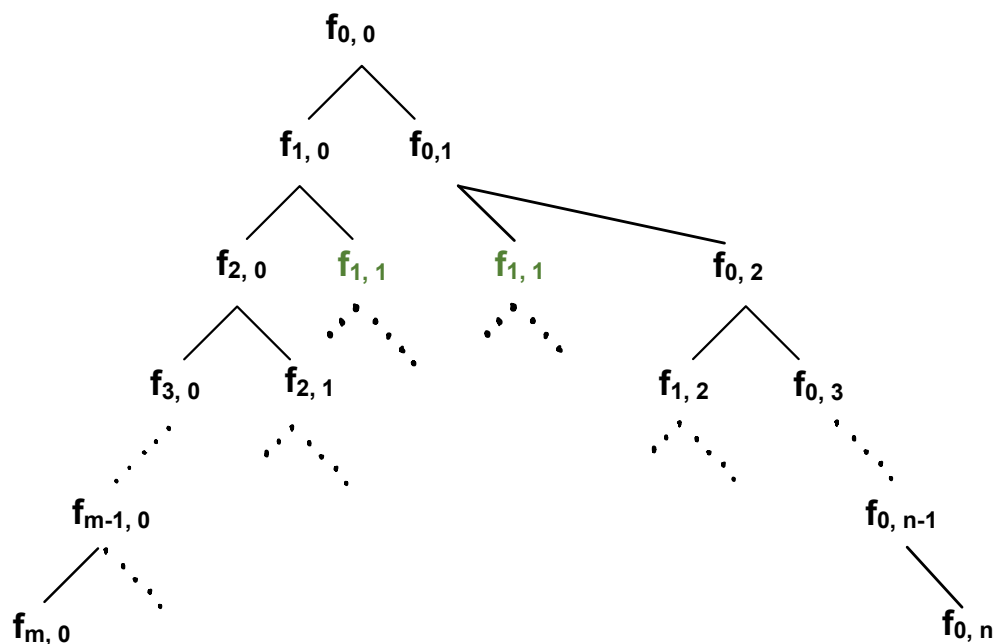
Recursive Case

Let $f(i, j)$ be a function which computes the **minimum initial health** the knight needs starting from the $(i, j)^{\text{th}}$ cell to **rescue the princess** from the $(m-1, n-1)^{\text{th}}$ cell then,

$$f(i, j) = \begin{cases} \min(x, y) & ; \text{grid}[i][j] == 0 \\ \min(x, y) - \text{grid}[i][j] & ; \text{grid}[i][j] < 0 \\ \min(x, y) - \text{grid}[i][j] & ; \text{grid}[i][j] > 0 \ \&\& \ \text{grid}[i][j] < \min(x, y) \\ 1 & ; \text{grid}[i][j] > 0 \ \&\& \ \text{grid}[i][j] \geq \min(x, y) \end{cases}$$

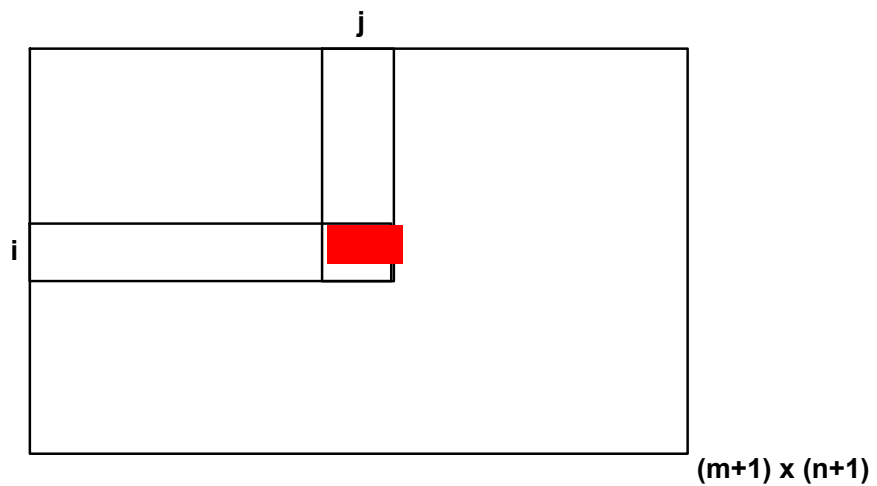
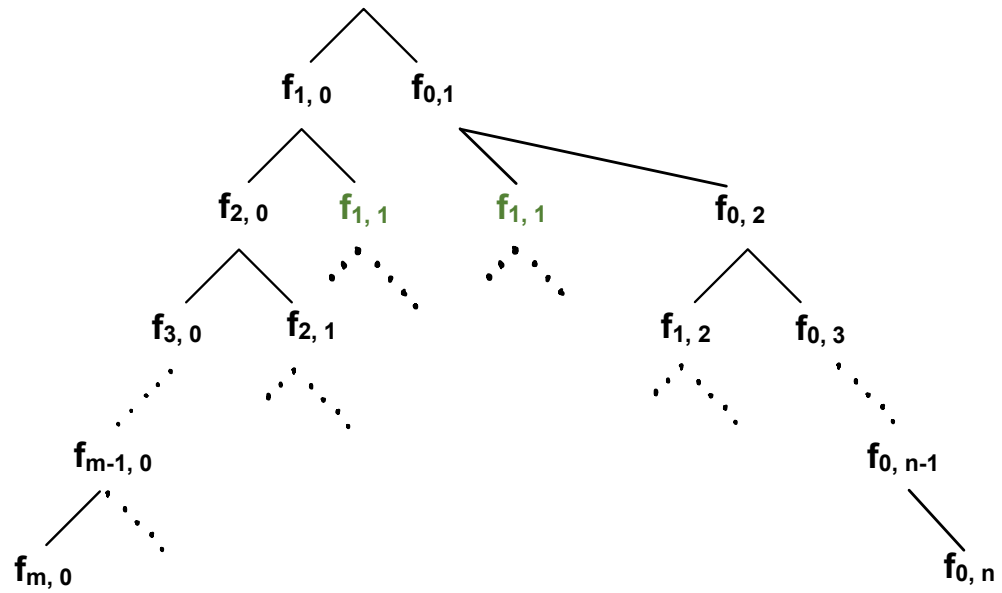
Base Case

What is the reason behind **exponential** time complexity ?



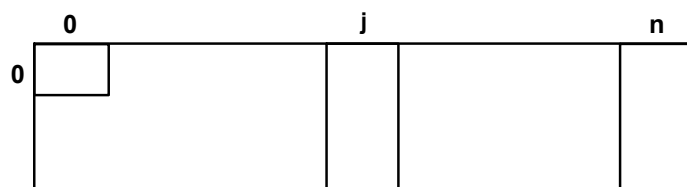
Top-Down Dynamic Programming

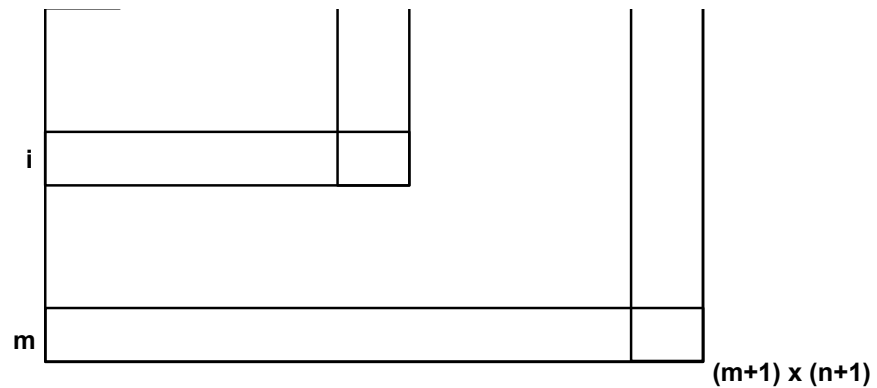
$f_{0,0}$



where,
 $f(i, j)$ is a function which computes the **minimum initial health** the knight requires starting from the $(i, j)^{\text{th}}$ cell to **rescue the princess**

Bottom-Up Dynamic Programming

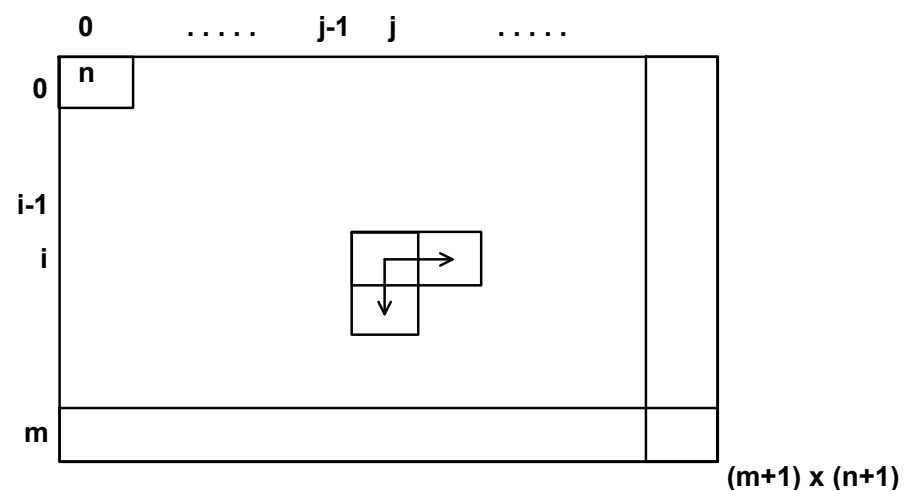




where,
 $f(i, j)$ is a function which computes the **minimum initial health** the knight requires starting from the **(i, j)th** cell to **rescue the princess**

From recurrence,

$$f(i, j) = \begin{cases} \min(x, y) & ; \text{grid}[i][j] == 0 \\ \min(x, y) - \text{grid}[i][j] & ; \text{grid}[i][j] < 0 \\ \min(x, y) - \text{grid}[i][j] & ; \text{grid}[i][j] > 0 \ \&\& \ \text{grid}[i][j] < \min(x, y) \\ 1 & ; \text{grid}[i][j] > 0 \ \&\& \ \text{grid}[i][j] \geq \min(x, y) \end{cases}$$



|-----| (m+1) x (n+1)

Example

Input : N = 5

	0	1	2	3	4
prices	2	3	5	1	4

Output : 50

1st year =

2nd year

3rd year

4th year

5th year

$$1 \cdot 2 = 2$$
$$2 \cdot 3 = 6$$
$$3 \cdot 4 = 12$$
$$4 \cdot 1 = 4$$
$$5.5 = 25$$

49

f_{ij} $\Rightarrow$ find max. profit from $\underbrace{\text{wines } [1 \dots j]}_{\text{in the } j^{\text{th}} \text{ year}}$ s.t we are j^{th}

```
init i = 0
j = n-1
y = 2
```

$$\underline{\underline{f_{0,n-1,1}}}$$
 $y = 2$

when $i=j$ you are left with a single bottle }

P: n

$$\max \left(\Rightarrow \overline{y_i + f_{i+1,j,y+1}} \right)$$
$$\Rightarrow y p_j + f_{i,j-1,y+1}$$
$$f_{ijy} =$$

Recursive Case

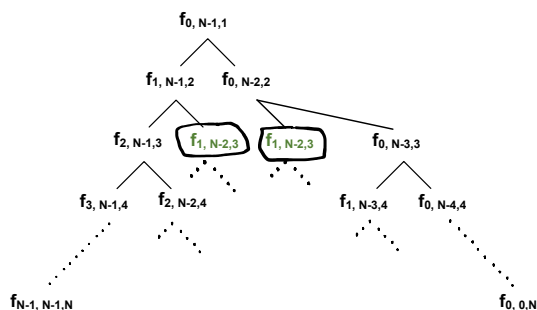
Let $f(i, j, y)$ be a function which computes, the **maximum profit** one can make by selling $(j-i+1)$ bottles of **wines** $[i...j]$ starting from the y^{th} year then,

$$f(i, j, y) = \max \{ p_i * y + f(i+1, j, y+1), p_j * y + f(i, j-1, y+1) \}$$

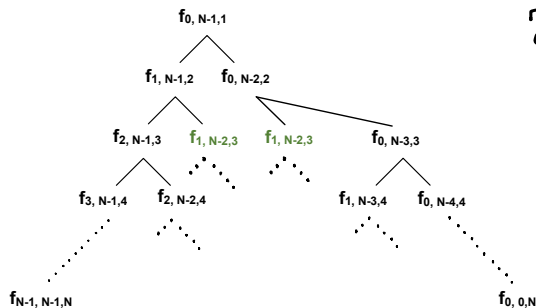
Base Case

if $\underbrace{i=j}_{\text{or } y=n}$ then $p_i \neq \underbrace{n}_y$ or $p_j \neq \underbrace{n}_y$

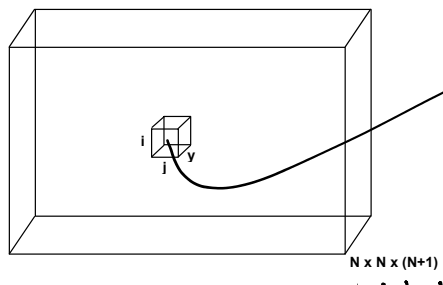
What is the reason behind **exponential** time complexity ?



Top-Down Dynamic Programming



$$\begin{aligned} 0 \leq i \leq n-1 &\rightarrow n \\ 0 \leq j \leq n-1 &\rightarrow n \\ 1 \leq y \leq n &\rightarrow n \\ \hline &n^3 \end{aligned}$$



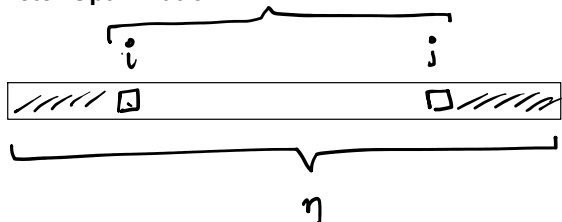
init: -1

$$dp_{i,j,y} = f_{i,j,y}$$

where,

$f(i, j, y)$ is a fn. which finds the **max. profit** one can make by selling $(j-i+1)$ bottles of wines $[i...j]$ starting from the y^{th} year.

State or Parameter Optimization



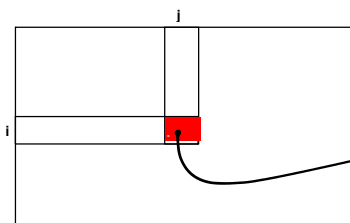
current year
 y^{th}

$$\begin{aligned} \# \text{ bottles left} &= j - i + 1 \\ \Rightarrow \# \text{ bottles sold} &= \eta - (j - i + 1) \end{aligned}$$

or
years passed

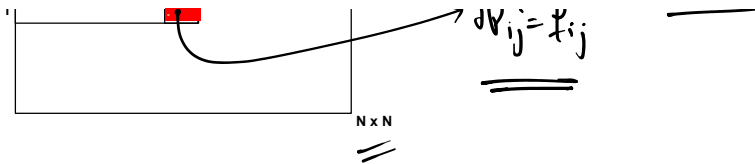
$$\begin{aligned} \text{current year } (y) &= \eta - (j - i + 1) + 1 \\ &= \eta - j + i - 1 + 1 \\ \hline y &= \eta - j + i \end{aligned}$$

init: -1



$$\begin{aligned} 0 \leq i \leq n-1 &\rightarrow n \\ 0 \leq j \leq n-1 &\rightarrow n \\ \hline &n \times n \end{aligned}$$

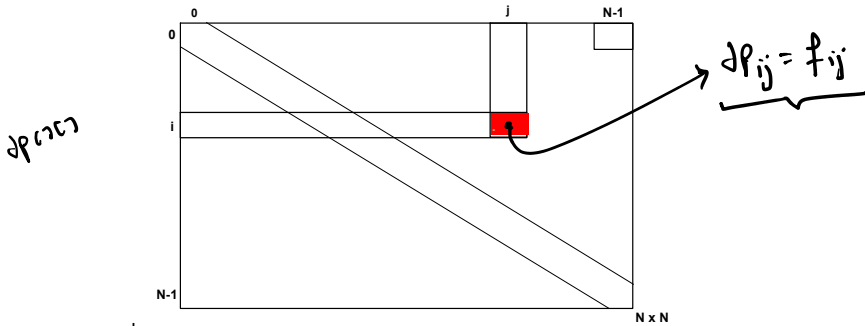
$$dp_{i,j} = f_{i,j}$$



where,

$f(i, j)$ is a fn. which finds the **max. profit** one can make by selling $(j - i + 1)$ bottles of **wines** $[i \dots j]$ starting from the y^{th} year such that $y = n - j + i$

Bottom Up Dynamic Programming



where,

$f(i, j)$ is a fn. which finds the **max. profit** one can make by selling $(j - i + 1)$ bottles of **wines** $[i \dots j]$ starting from the y^{th} year such that $y = n - j + i$

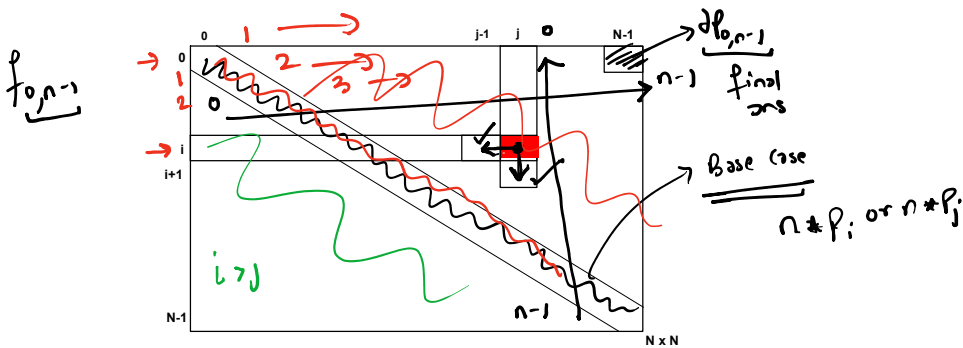
From recurrence,

$$f(i, j) = \begin{cases} \max \{ p_i * y + f(i+1, j), p_j * y + f(i, j-1) \} & ; i < j \\ p_i * y & ; i = j \end{cases}$$

$y = n - j + i$

Therefore,

$$dp[i][j] = \begin{cases} \max \{ p_i * y + dp[i+1][j], p_j * y + dp[i][j-1] \} & ; i < j \\ p_i * y & ; i = j \end{cases}$$



0/1 Knapsack Problem

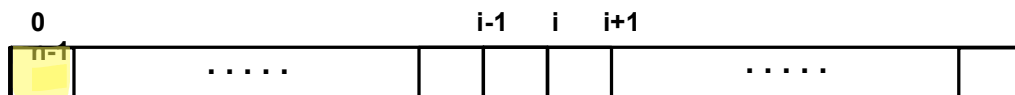
Given a set of **N items** along with their **prices** $\{ p_0, p_1, \dots, p_{N-1} \}$ and **weights** $\{ w_0, w_1, \dots, w_{N-1} \}$ & a knapsack that has a **maximum weight capacity** of **W**. Design an algorithm to find the **maximum profit** you can make by **selecting a subset of items** & placing them in the knapsack such that the **sum of weights** of the selected items does not exceed the knapsack's capacity.

Example

Input : **N = 5**, **W = 4**

	0	1	2	3	4
prices	8	4	1	5	3
weights	1	2	3	2	2

Output : **13**



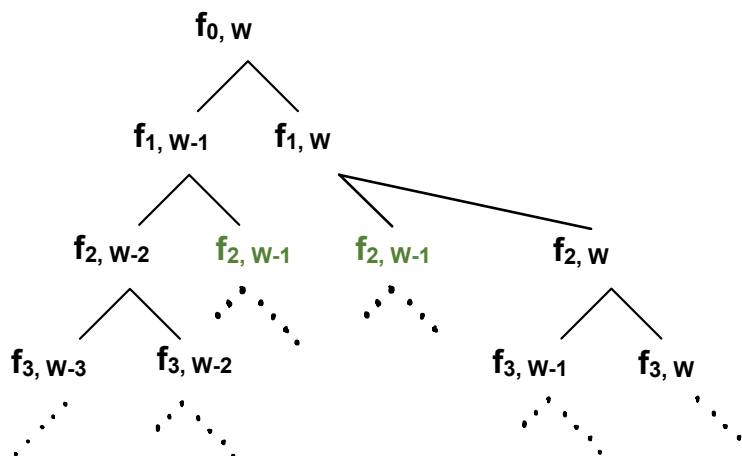
Recursive Case

Let $f(i, w)$ be a function that finds the **maximum profit** you can make by selecting a subset of **items** $[i \dots N-1]$ and placing them in the knapsack such that the current knapsack capacity is w then,

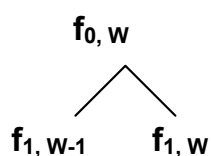
$$f(i, w) = \begin{cases} \max \{ p_i + f(i+1, w-w_i), f(i+1, w) \} & ; w_i \leq w \\ f(i+1, w) & ; \text{otherwise} \end{cases}$$

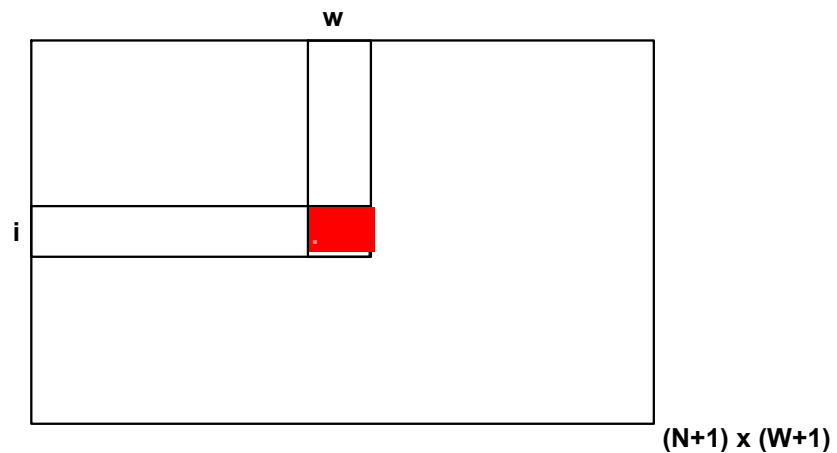
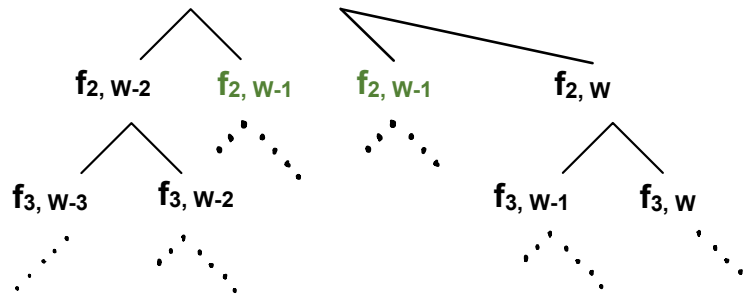
Base Case

What is the reason behind **exponential** time complexity ?



Top-Down Dynamic Programming

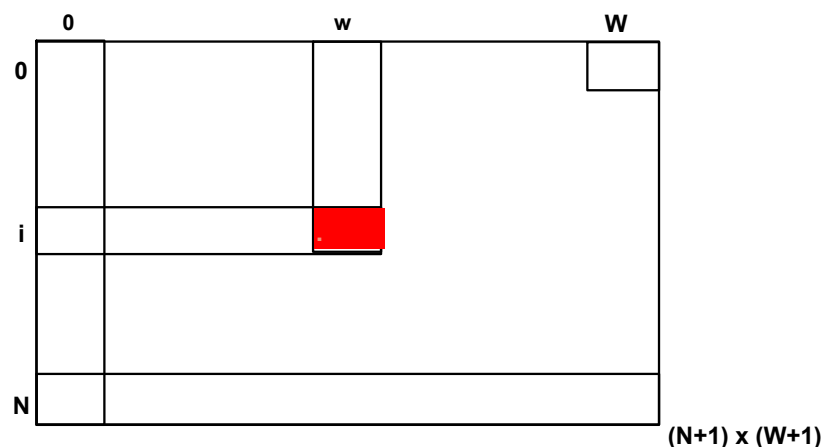




where,

$f(i, w)$ is a function that finds the **maximum profit** you can make by selecting a subset of **items** $[i \dots N-1]$ and placing them in the knapsack such that the current knapsack capacity is w

Bottom-Up Dynamic Programming



where,

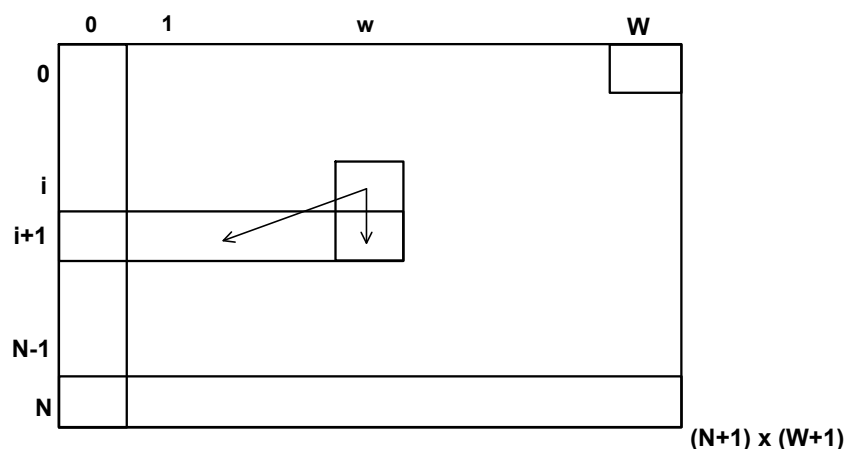
$f(i, w)$ is a function that finds the **maximum profit** you can make by selecting a subset of **items** $[i \dots N-1]$ and placing them in the knapsack such that the current knapsack capacity is w

From recurrence,

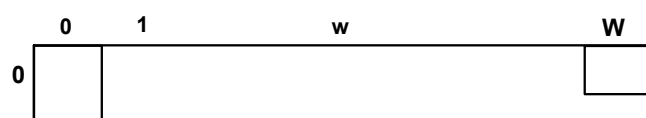
$$f(i, w) = \begin{cases} \max \{ p_i + f(i+1, w-w_i), f(i+1, w) \} & ; w_i \leq w \\ f(i+1, w) & ; w_i > w \\ 0 & ; i = n \text{ or } w = 0 \end{cases}$$

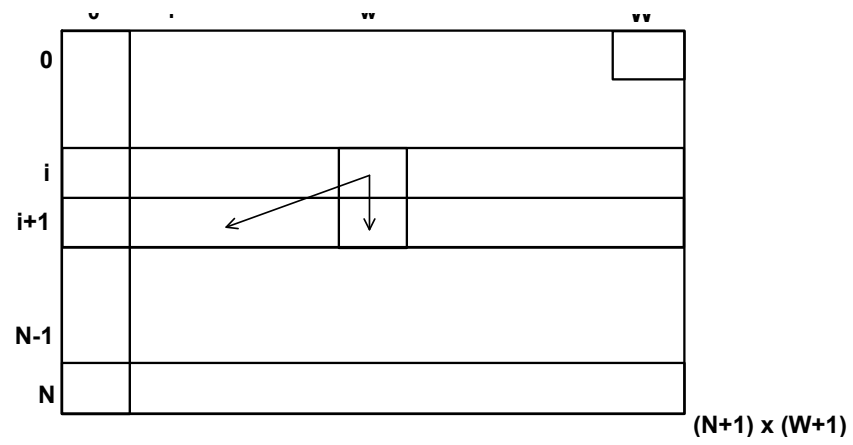
Therefore,

$$dp[i][w] = \begin{cases} \max \{ p_i + dp[i+1][w-w_i], dp[i+1, w] \} & ; w_i \leq w \\ dp[i+1][w] & ; w_i > w \\ 0 & ; i = n \text{ or } w = 0 \end{cases}$$



Bottom-Up Dynamic Programming Space Optimized





<https://www.youtube.com/watch?v=DK5Z9ttunEQ&t=4739s>

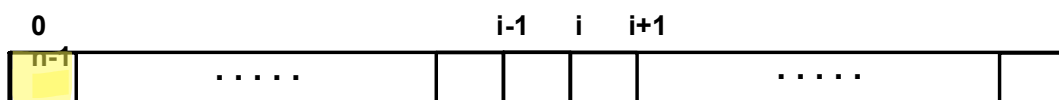
Unbounded Knapsack Problem

Given a set of N objects along with their **prices** $\{p_0, p_1, \dots, p_{N-1}\}$ and **weights** $\{w_0, w_1, \dots, w_{N-1}\}$ and a knapsack that has a maximum **weight capacity** of W . Design a recursive algorithm to compute the **maximum profit** one can make by choosing a subset of objects and putting them in the knapsack such that the sum of weights of the objects put in the knapsack **doesn't** exceed knapsack capacity and you can choose an item **multiple times**.

Example

Input : $N = 3$, $W = 10$

	0	1	2
prices	6	3	8
weights	3	1	4



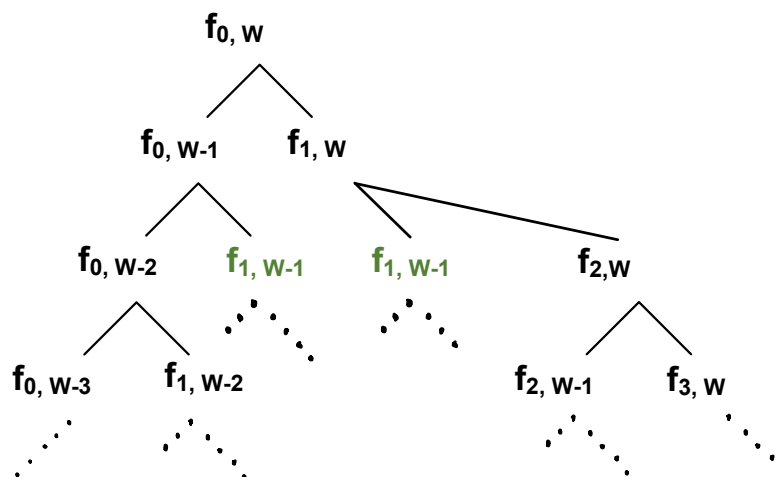
Recursive Case

Let $f(i, w)$ be a function which computes the **maximum profit** one can make by choosing a subset of objects to be put in the knapsack from the given set of objects that **starts** at the i^{th} index such that the knapsack capacity is w then,

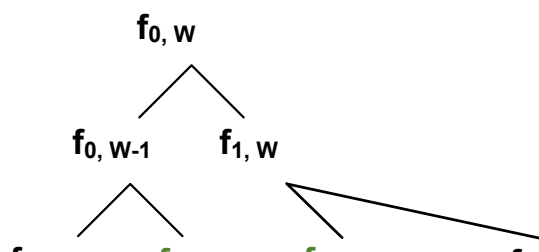
$$f(i, w) = \begin{cases} \max \{ p_i + f(i, w-w_i), f(i+1, w) \} & ; w_i \leq w \\ f(i+1, w) & ; \text{otherwise} \end{cases}$$

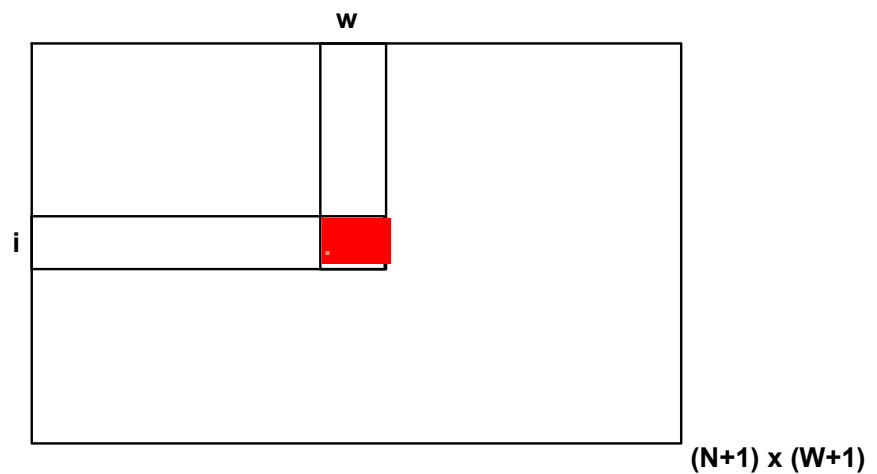
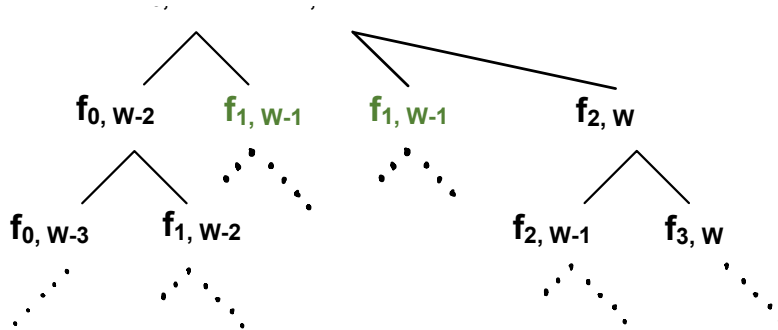
Base Case

What is the reason behind **exponential** time complexity ?



Top-Down Dynamic Programming

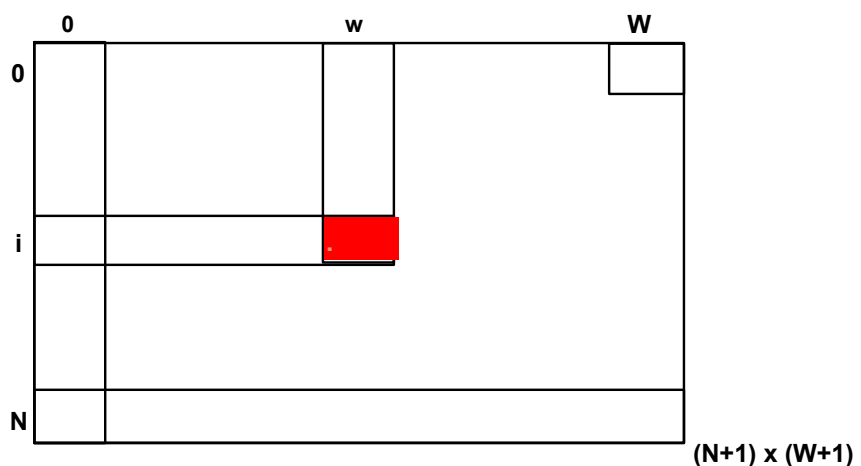




where,

$f(i, w)$ is a function which computes the **maximum profit** one can make by choosing a subset of objects to be put in the knapsack from the given set of objects starting at the i^{th} index such that the knapsack capacity is w .

Bottom-Up Dynamic Programming



where,

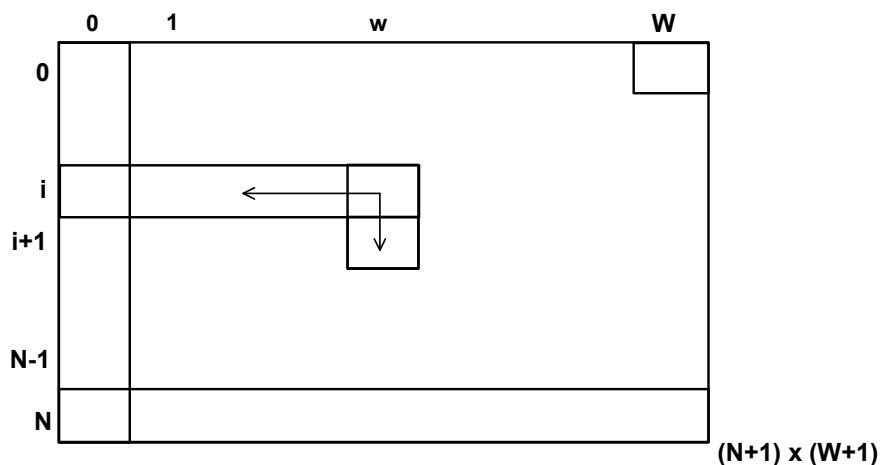
$f(i, w)$ is a function which computes the **maximum profit** one can make by choosing a subset of objects to be put in the knapsack from the given set of objects starting at the i^{th} index such that the knapsack capacity is w .

From recurrence,

$$f(i, w) = \begin{cases} \max \{ p_i + f(i, w-w_i), f(i+1, w) \} & ; w_i \leq w \\ f(i+1, w) & ; w_i > w \\ 0 & ; i = n \text{ or } w = 0 \end{cases}$$

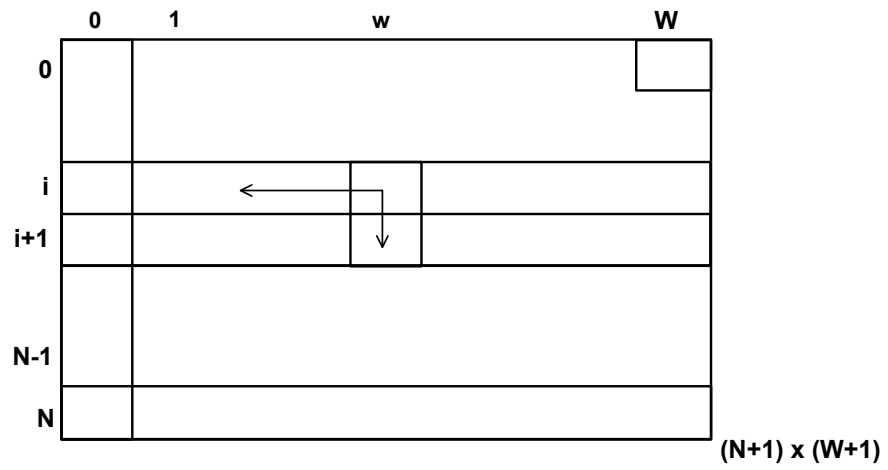
Therefore,

$$dp[i][w] = \begin{cases} \max \{ p_i + dp[i][w-w_i], dp[i+1, w] \} & ; w_i \leq w \\ dp[i+1][w] & ; w_i > w \\ 0 & ; i = n \text{ or } w = 0 \end{cases}$$



Bottom-Up Dynamic Programming Space Optimized

Bottom-Up Dynamic Programming Space Optimized



<https://www.youtube.com/watch?v=DK5Z9ttunEQ&t=6370s>

Colorful Knapsack Problem

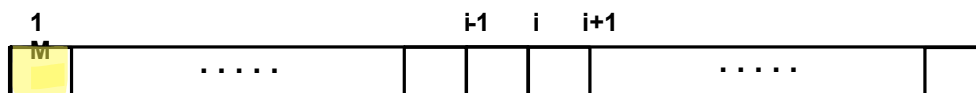
You are given a sequence of **N** objects that are coloured using **M** colours labelled from **1** to **M**, along with their **weights**, and a knapsack that has a weight **capacity** of **X**. You are required to fill the knapsack with the given set of objects in such a way that its **unused** capacity is **minimised**. Design an algorithm to find the **minimum unused capacity** of the knapsack such that the knapsack contains exactly **one** object of each colour and the sum of the weights of objects put in the knapsack **doesn't exceed** its capacity.

note : return -1 if there is no way to fill the knapsack

Example

Input : **N = 8, M = 3, X = 13**

	0	1	2	3	4	5	6	7
weights	2	3	4	2	4	5	2	3
colors	1	1	1	2	2	2	3	3



Recursive Case

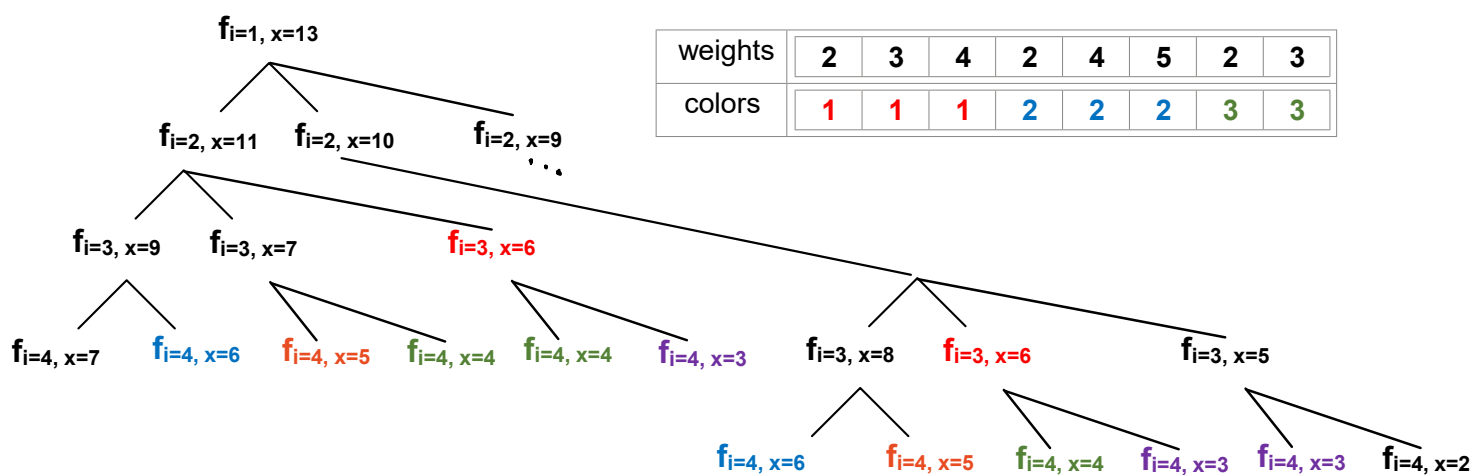
Let **f(i, x)** be a function which computes the **minimum unused capacity** of

the **knapsack** when we choose a **sequence of objects** to be put in the knapsack, one object from each of the remaining colors, starting with the i^{th} **color** such that the current capacity of the knapsack is x then,

$$f(i, x) = \min \{ f(i+1, x-x_{ij}) \} ; x_{ij} \leq x$$

Base Case

What is the reason behind **exponential** time complexity ?



Top-Down Dynamic Programming

Longest Common Subsequence (LCS)

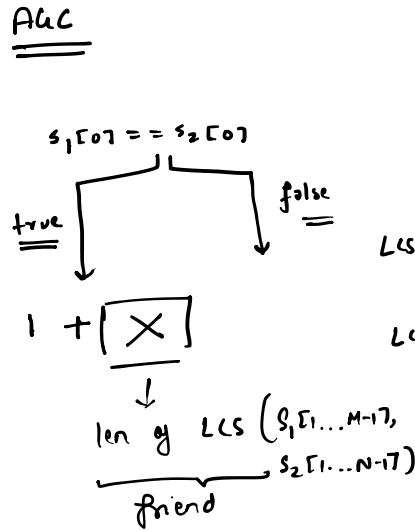
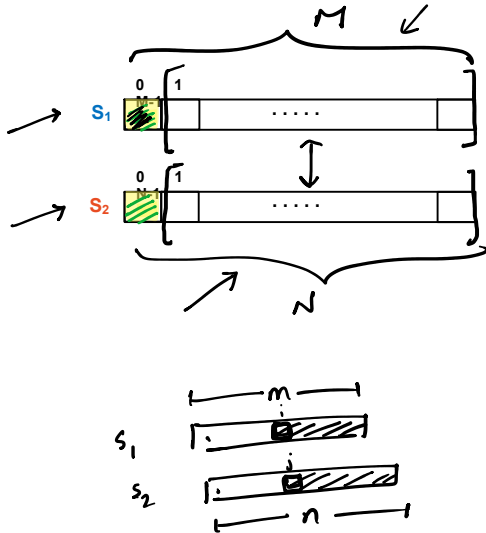
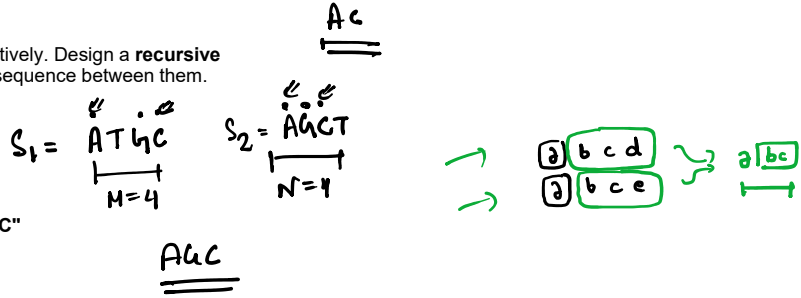
Given two strings S_1 and S_2 of lengths M and N respectively. Design a **recursive** algorithm to find the **length** of the **longest** common subsequence between them.

Example :

Input : $S_1 = \text{"ATGC"} , M = 4$
 $S_2 = \text{"AGCT"} , N = 4$

Output : 3

Explanation : The LCS between S_1 and S_2 is "AGC"



$$\left. \begin{aligned} \text{LCS}(S_1[1 \dots M-1], S_2[0 \dots N-1]) &\rightarrow \gamma \\ \text{LCS}(S_1[0 \dots M-1], S_2[1 \dots N-1]) &\rightarrow \delta \end{aligned} \right\} \max$$

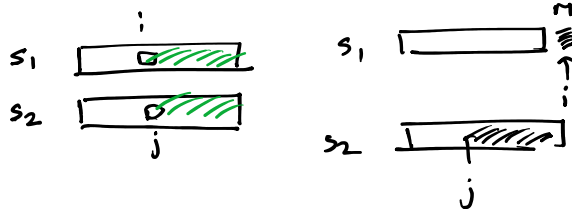
Recursive Case

Let $f(i, j)$ be a function which computes the **length** of the **longest common subsequence** between $S_1[i \dots M-1]$ and $S_2[j \dots N-1]$ then,

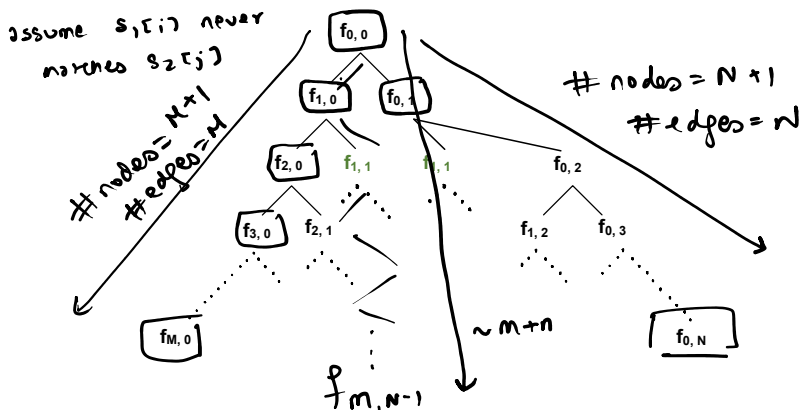
$$f(i, j) = \begin{cases} \max \{ f(i+1, j), f(i, j+1) \} & ; S_1[i] \neq S_2[j] \\ 1 + f(i+1, j+1) & ; S_1[i] = S_2[j] \end{cases}$$

Base Case

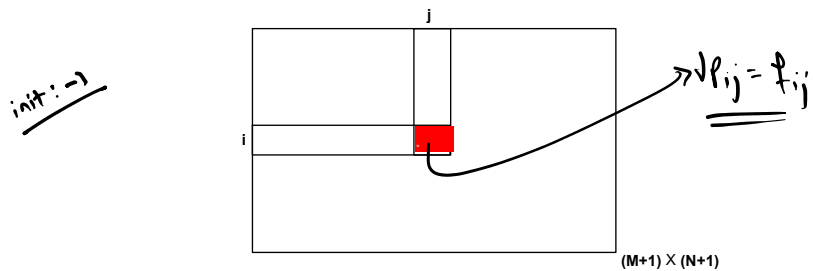
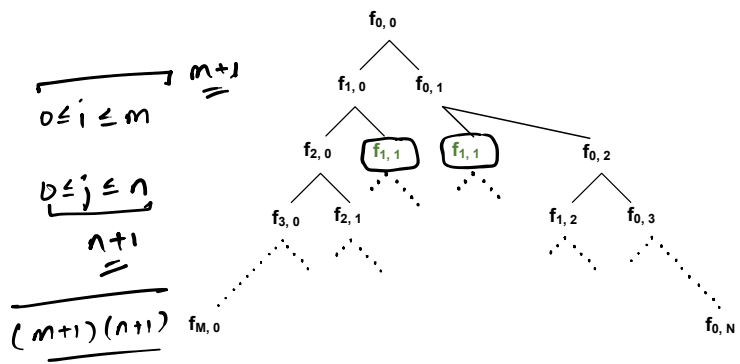
$$\begin{aligned} f(i=0, j=0) &= 0 \\ f(i=M, j) &= 0 \\ f(i, j=N) &= 0 \end{aligned}$$



What is the reason behind **exponential** time complexity ?



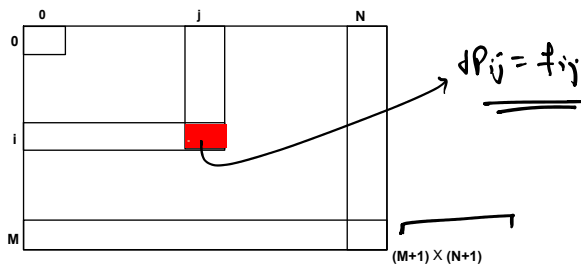
Top-Down Dynamic Programming



where,

$f(i, j)$ is a function which computes the **length** of the **longest common subsequence** between $S_1[i \dots M-1]$ and $S_2[j \dots N-1]$

Bottom Up Dynamic Programming



where,

$f(i, j)$ is a function which computes the **length** of the **longest common subsequence** between $S_1[i \dots M-1]$ and $S_2[j \dots N-1]$

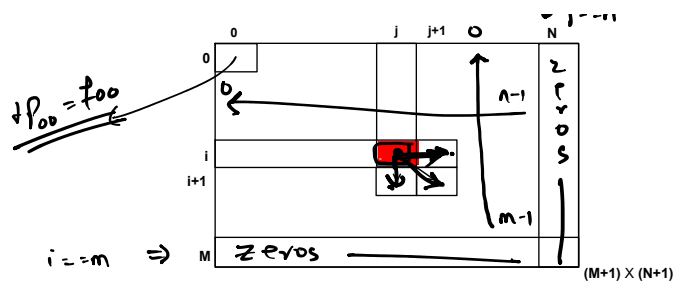
From recurrence,

$$f(i, j) = \begin{cases} \max \{ f(i+1, j), f(i, j+1) \} & ; S_1[i] \neq S_2[j] \\ 1 + f(i+1, j+1) & ; S_1[i] == S_2[j] \end{cases}$$

Therefore,

$$dp[i][j] = \begin{cases} \max \{ dp[i+1][j], dp[i][j+1] \} & ; S_1[i] \neq S_2[j] \\ 1 + dp[i+1][j+1] & ; S_1[i] == S_2[j] \end{cases}$$



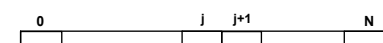
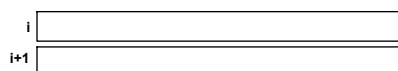
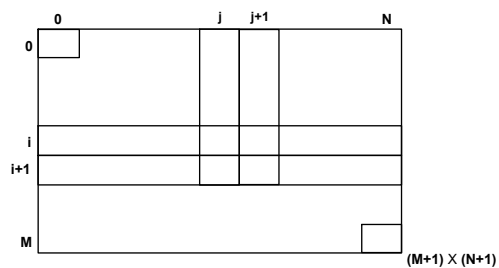


[R][HW] Building the LCS from dp[][]

		A	G	C	T	
		0	1	2	3	4
A	0	3	2	1	1	0
T	1	2	2	1	1	0
G	2	2	2	1	0	0
C	3	1	1	1	0	0
	4	0	0	0	0	0

dp[][]

[R][HW] Space Optimized Bottom-Up Dynamic Programming



	0		j	j+1		N
0						
i						
i+1						
M						

$(M+1) \times (N+1)$

Edit Distance

Given two strings S_1 and S_2 of lengths M and N respectively. Design a **recursive** algorithm to find the **minimum** number of **edits** required to transform S_1 into S_2 such that each **edit operation** performed on S_1 can be a letter **insertion** or letter **deletion** or a letter **substitution**.

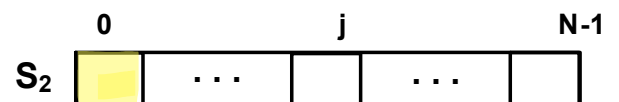
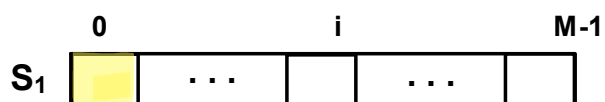
Example :

Input : $S_1 = \text{"food"}$; $S_2 = \text{"money"}$

Output : 4

0	1	2	3
f	o	o	d

0	1	2	3	
4				
m	o	n	e	y



Recursive Case

Let $f(i, j)$ be a function which computes the **minimum** no. of edits required to transform $S_1[i \dots M-1]$ into $S_2[j \dots N-1]$ then,

$$f(i, j) = \min \begin{cases} 1 + f(i, j+1), \\ 1 + f(i+1, j), \\ S_1[i] \neq S_2[j] + f(i+1, j+1) \end{cases}$$

Base Case

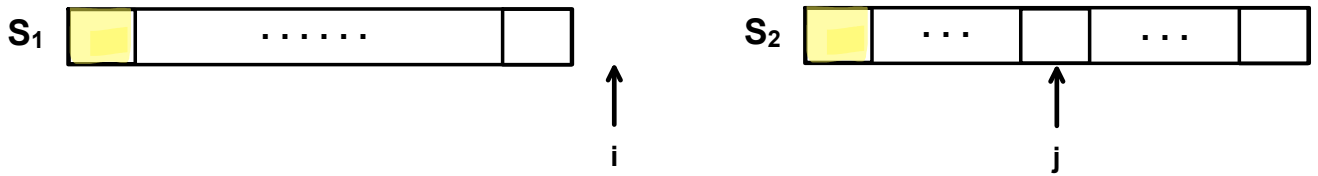
$f(i=M, j)$ is a function which computes the **minimum** number of **edits** required to transform $S_1[M \dots M-1]$ into $S_2[j \dots N-1]$

0

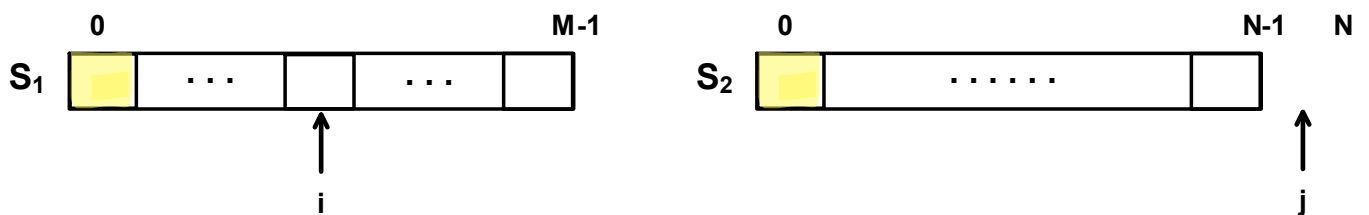
M-1 M

0

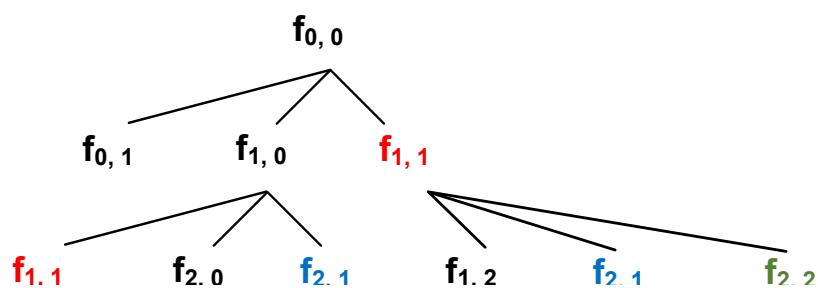
N-1

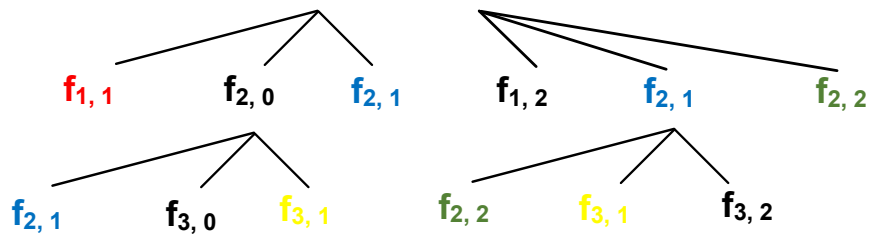


$f(i, j=N)$ is a function which computes the **minimum** number of **edits** required to transform $S_1[i \dots M-1]$ into $S_2[N \dots N-1]$

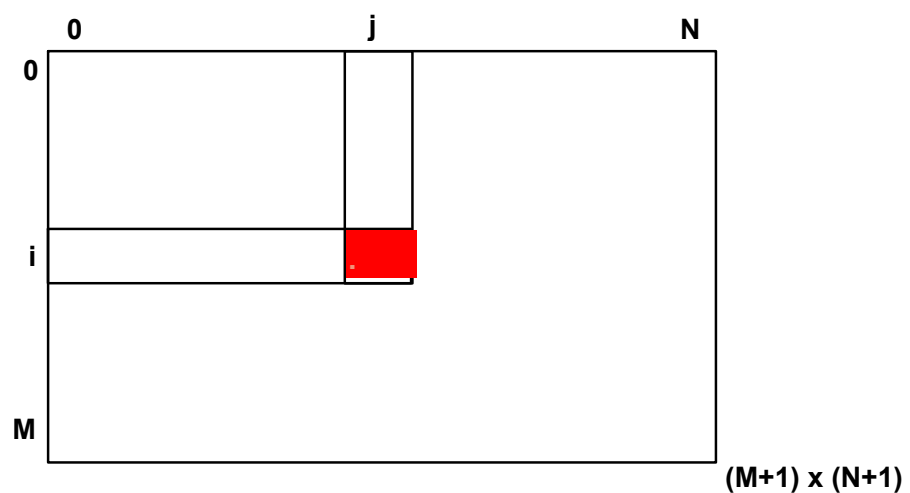
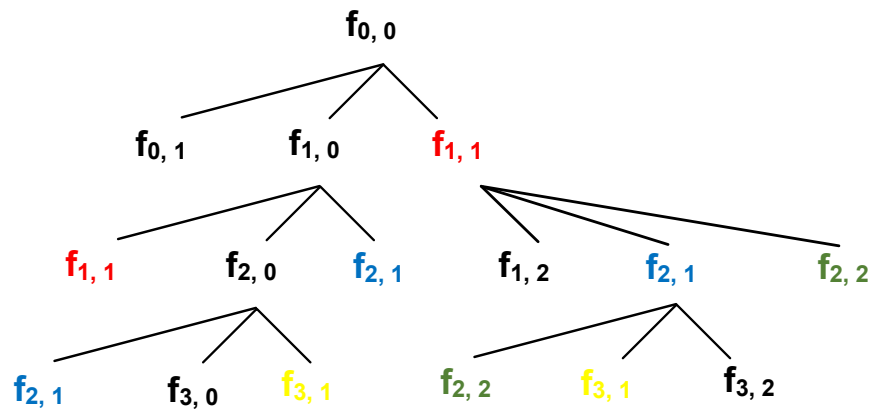


What is the reason behind **exponential** time complexity ?





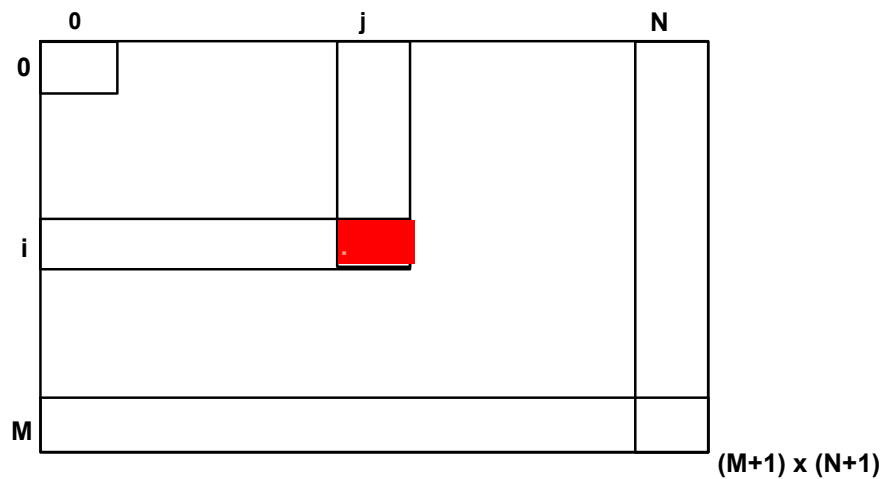
Top-Down Dynamic Programming



where,

$f(i, j)$ is a function which computes the **minimum number** of **edits** required to transform $S_1[i \dots M-1]$ into $S_2[j \dots N-1]$

Bottom Up Dynamic Programming



where,

$f(i, j)$ is a function which computes the **minimum number** of **edits** required to transform $S_1[i \dots M-1]$ into $S_2[j \dots N-1]$

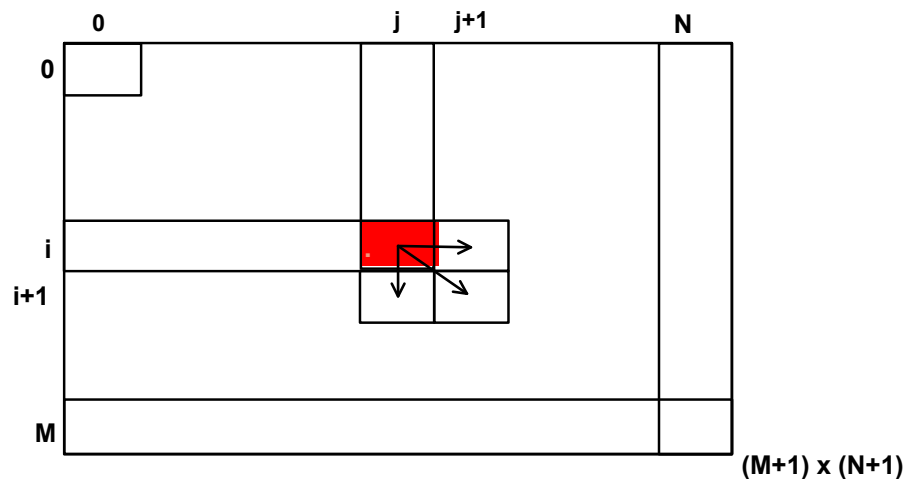
From recurrence,

$$f(i, j) = \min \begin{cases} 1 + f(i, j+1), \\ 1 + f(i+1, j), \\ S_1[i] \neq S_2[j] + f(i+1, j+1) \end{cases}$$

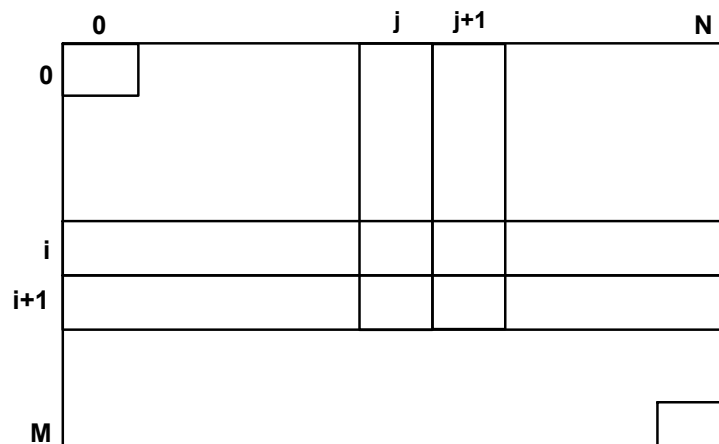
Therefore,

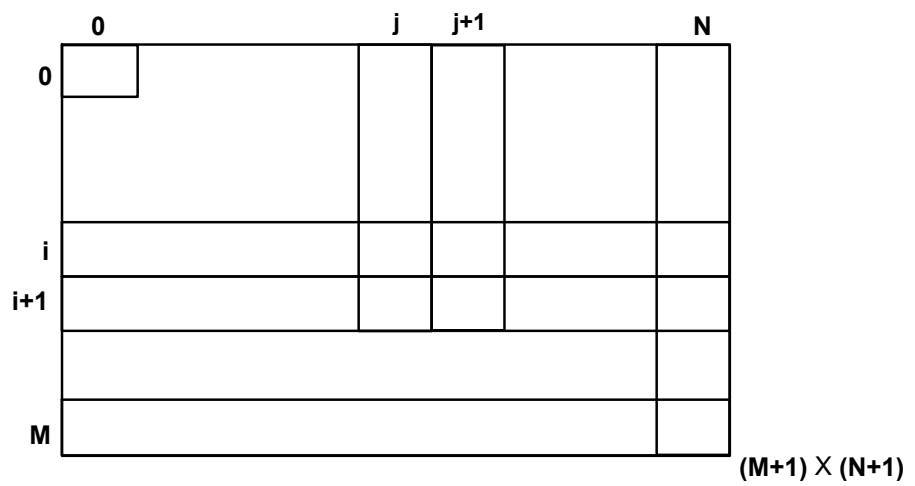
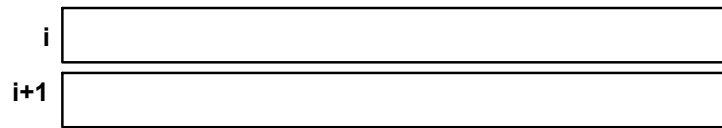
$$dp[i][j] = \min \begin{cases} 1 + dp[i][j+1], \\ 1 + dp[i+1][j], \\ S_1[i] \neq S_2[j] + dp[i+1][j+1] \end{cases}$$

$$dp[i][j] = \min \begin{cases} dp[i][j+1], \\ 1 + dp[i+1][j], \\ S_1[i] \neq S_2[j] + dp[i+1][j+1] \end{cases}$$



Space Optimized Bottom-Up Dynamic Programming





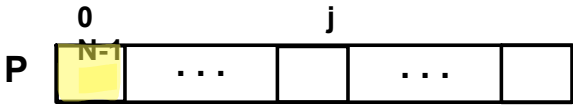
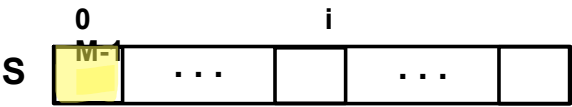
Wildcard Matching

Given an input string **S** and a pattern **P** that contains 0 or more **wildcard characters** ('?' and '*'), design an algorithm to determine if **P completely matches S** such that the wildcard '?' can match any single character and '*' matches 0 or more characters.

Example

S	a	b	c	a	b	c	a	a
P	a	?	c	*	?	a	*	

S	a	b	c	a	b	c
P	a	?	b	*		



Recursive Case

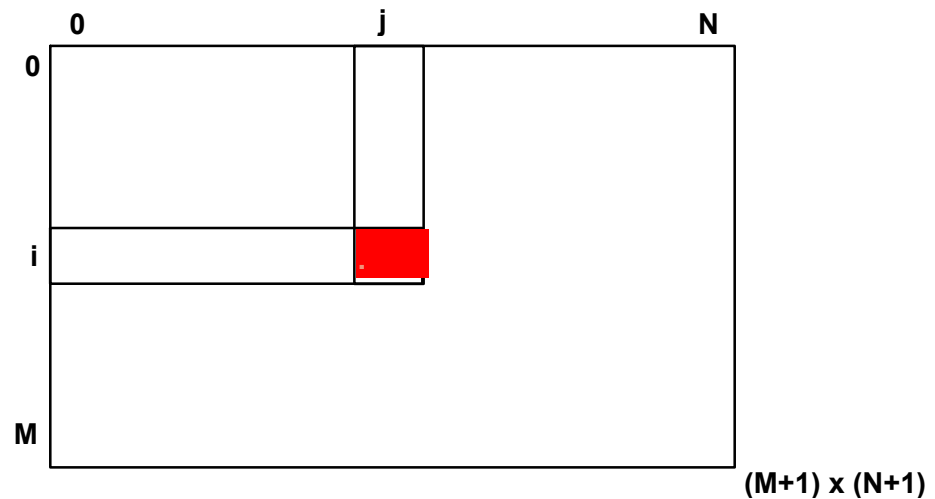
Let $f(i, j)$ be a function which checks if $P[j \dots N-1]$ matches $S[i \dots M-1]$ then,

$$f(i, j) = \begin{cases} f(i+1, j+1) & ; P[j] == S[i] \text{ or } P[j] == '?' \\ f(i, j+1) \text{ or } f(i+1, j) & ; P[j] == '*' \\ \text{false} & ; P[j] != S[i] \end{cases}$$

Base Case

What is the reason behind **exponential** time complexity ?

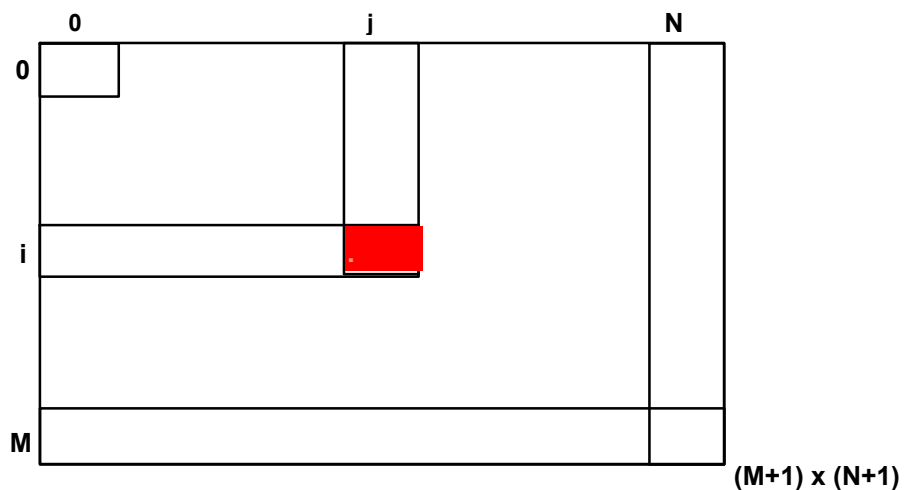
Top-Down Dynamic Programming



where,

$f(i, j)$ is a function which checks if $P[j \dots N-1]$ matches $S[i \dots M-1]$

Bottom Up Dynamic Programming



where,

$f(i, j)$ be a function which checks if $P[j \dots N-1]$ matches $S[i \dots M-1]$

From recurrence,

$$f(i, j) = \begin{cases} f(i+1, j+1) & ; P[j] == S[i] \text{ or } P[j] == '?' \\ f(i, j+1) \text{ or } f(i+1, j) & ; P[j] == '*' \\ \text{false} & ; P[j] != S[i] \end{cases}$$

Therefore,

$$dp[i][j] = \begin{cases} dp[i+1][j+1] & ; P[j] == S[i] \text{ or } P[j] == '?' \\ dp[i][j+1] \text{ or } d[i+1][j] & ; P[j] == '*' \\ \text{false} & ; P[j] != S[i] \end{cases}$$

